Excise: One-Shot Capability Extraction from Open Language Models

Arya G. M.
arya@e3group.ai

June 2026 *(workshop preprint; artifact:* `pip install excise`*)*

Abstract

A deployed language model usually exercises one capability while paying for all of them. Recent work (PRISM) showed that a single capability can run through a sparse subset of MLP channels, but found it with a staged pipeline: train a behavior-preserving adapter, attribute channels, select a mask, and re-train under it—each stage a separate run, with the budget frontier mapped point by point. We collapse this into a single training run: learnable hard-concrete gates on every MLP channel train *jointly* with a low-rank adapter under a forward-KL leash to the frozen base model, while an adaptive controller lowers the sparsity target as fast as behavior permits and *generation probes*—not loss curves—decide when the floor is reached. On PRISM's arithmetic benchmark our single run reaches **100.9% recovery at 5% of MLP channels** and finds a **2.9%-channel floor at 97% recovery** automatically, surpassing the staged pipeline's hand-tuned best (91.3% at 5.05%; refined minimum 4.65% at 90.6%) on both axes. The method is label-free (the target is the model's own output distribution), generalizes to function calling on Qwen3-4B (76% verbatim output reproduction at 40% of channels, against a 19% baseline at comparable budgets), and exports a physically smaller model: slicing the masked channels shrinks Qwen2.5-Math-1.5B to 0.42B parameters (3.7×) with no fidelity loss relative to masking. We also report two instructive failures: distribution-level KL is silently miscalibrated at high sparsity (it reads healthy while true recovery collapses by 35 points), and whole-layer gating lets the adapter overfit the training distribution while generalization fails. Both shaped the method. Everything runs on a single consumer GPU; total compute for all results in this paper was under $5.

1 Introduction

Model efficiency methods—pruning, distillation, quantization—ask how much of a network can be removed while *aggregate* performance survives. Mishra and Pagare [12] asked a sharper question: given one named behavior, how small a substrate of the network can carry it once everything else is switched off? Their answer, the PRISM framework, established what we adopt wholesale and credit as the foundation of this work: the *extraction contract* (a channel subset counts only if behavior survives with the complement deactivated), *recovery* as the metric, and the central finding that a behavior-preserving adaptation step ("collimation") can relocate a capability into far fewer channels than attribution alone suggests.

PRISM's pipeline, however, is staged: collimate, attribute with a relevance method, select a top-k mask, optionally re-collimate under the frozen mask. The stages create the central limitation their paper documents: an adapter trained *after* mask selection provably cannot shrink the mask, and one trained *before* can only move the frontier indirectly. The budget k is swept manually; each frontier point costs evaluations; the minimal circuit is found by hand-refinement. Their stated highest-value direction is to reshape the budget *during* training with an L_0 or gating objective.

We build exactly that, and package it as a tool. Our contributions:

1. **One-shot joint extraction.** Hard-concrete gates [10] on all MLP intermediate channels train jointly with a LoRA adapter [9] under a mass-covering KL constraint to the frozen base. Selection and relocation negotiate continuously; the regime split between "collimate-then-attribute" and "attribute-then-collimate" dissolves (§2).

2. **Adaptive floor discovery with behavior probes.** A Lagrangian controller lowers the sparsity target whenever a KL budget is satisfied, and cheap greedy-generation probes veto the descent when the model's actual outputs degrade. The full sparsity–fidelity frontier and its floor come from one run (§2).
3. **Label-free operation.** The distillation target is the base model's own greedy output and full output distribution; no labeled data is required (§3).
4. **Physical export.** For gated MLPs, deleting masked channels is exactly equivalent to zero-masking them. We verify this empirically and ship it: 1.54B → 0.42B parameters at 97% recovery (§3.3).
5. **Two calibrated negative results.** Distribution-level KL is an unreliable stopping signal at high sparsity, and layer-granular gates permit silent train-distribution overfitting (§4). We argue these matter beyond our method: faithfulness metrics built on logit divergence inherit the same blind spot.

2 Method

Setup. Let M be a frozen decoder-only transformer and $\mathcal{C}$ its MLP intermediate channels ($L \times d_{\text{ff}}$; the inputs to each block's down-projection), following PRISM's unit of extraction. Each channel c gets a hard-concrete gate $z_c \in [0, 1]$ parameterized by $\log \alpha_c$ [10, 11]; a LoRA adapter θ (rank 32, all linear modules) provides the relocation capacity. Given prompts x exercising the target capability, the teacher is the base model itself: greedy continuations $\hat{y} = M(x)$ and the full distribution $p_M(\cdot \mid x, \hat{y}_{<t})$ at each output position, cached once as renormalized top-128 sparse vectors.

Objective. Each step samples gates z (outside any gradient-checkpointed region; see §4) and minimizes

$$\mathcal{L} = \underbrace{\mathrm{KL}(p_M \,\|\, p_{\theta,z})}_{\text{masked student tracks base}} + \beta_{\text{ce}}\,\mathrm{CE}(\hat{y}) \;+\; \lambda\,\mathbb{E}[\|z\|_0]\,/|\mathcal{C}| \;+\; \beta_g \underbrace{\mathrm{KL}(p_M \,\|\, p_\theta)}_{\text{guardrail (every 4th step)}}, \tag{1}$$

with β_{ce}=0.05, β_g=1. The forward (mass-covering) KL direction follows PRISM's collimation analysis: the student may not drop probability anywhere the base places it. The guardrail evaluates the *unmasked* adapted model and anchors it to the base, preventing the adapter from "solving the task masked" by becoming a different model; without it, unmasked accuracy silently drifted 3.7 points in our early runs, and with it the drift is zero (§3).

Adaptive controller. A target open fraction τ starts at 1 and decays multiplicatively ($\times$0.993 per step) whenever the EMA of the distillation KL is under a budget (0.02–0.025 in all experiments); the Lagrange multiplier λ follows $\lambda \leftarrow \max(0, \lambda + \eta\,(\bar{z} - \tau))$. Once the open fraction falls below a threshold, every 150 steps a *probe* hardens the current top-k mask and greedily generates on a small training subset; if exact-match against the teacher outputs degrades beyond tolerance twice consecutively, the floor is declared. After the descent stops, the mask is frozen at the floor budget and the adapter alone is polished briefly under the exact binary mask, removing the stochastic-gate train/eval mismatch. Final evaluation hardens top-k masks at standard budgets and reports held-out fidelity, yielding the frontier from this single run.

Receipts. Every run reports a same-size random-mask control (should be ≈0), unmasked drift (should be ≈0), and the probe trace. These guard the two failure modes we observed and one we inherited from PRISM's analysis: trivial tasks, cheating adapters, and masks that only correlate with behavior.

3 Results

All experiments ran on one RTX 4090 ($0.40/hr); the complete set below consumed under $5. Code, configs, and receipts: `github.com/e3group/excise`.

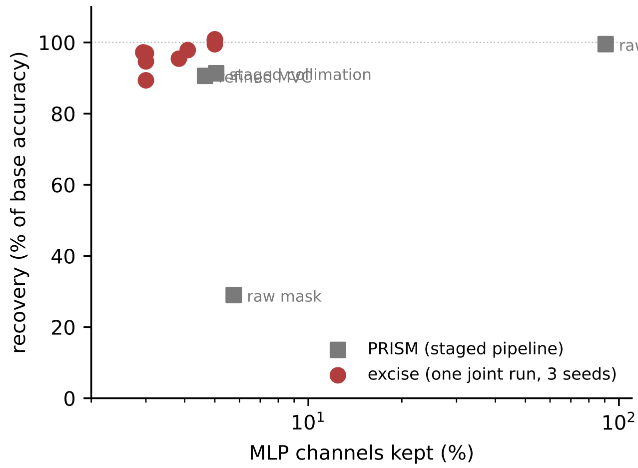

Figure 1: Sparsity–recovery frontier, 2-digit addition, Qwen2.5-Math-1.5B. Gray squares: PRISM's staged pipeline as reported [12]. Red circles: one `excise` run per seed (3 seeds), which emits the whole frontier; floors (2.9–4.1%) were found by the controller, not swept. Caveats for this comparison in §5.

3.1 Arithmetic: surpassing the staged pipeline on its benchmark

We use PRISM's testbed: 2-digit addition on Qwen2.5-Math-1.5B [15] (canonical prompt `"a + b ="`, exhaustive sums, 10% held out; 250,880 MLP channels), zero-isolation deactivation, and recovery = masked accuracy / base accuracy (base: 97.4%).

	channels kept	recovery
PRISM raw attribution mask	5.75%	29.0%
PRISM staged collimation	5.05%	91.3%
PRISM refined minimal circuit (manual)	4.65%	90.6%
random mask control (ours)	5%	0.0%
grad×act attribution mask (ours)	5%	0.3%
excise, one run (seed 42 / 43 / 44)	5%	**100.9 / 100.4 / 99.5%**
excise, automatic floor	**2.9 / 3.8 / 4.1%**	97.2 / 95.4 / 97.9%

Table 1: Arithmetic extraction. Every seed beats the staged pipeline at 5%, and every automatically-found floor is at or below the hand-refined minimal circuit with higher recovery. The unmasked guardrail held base accuracy exactly (97.4%) in all runs.

Ablations (Figure 2): restricting the adapter to MLP projections (the strictest frozen-scaffold contract) costs ∼3 points at 5% and runs 25% faster; removing labels entirely (pure self-distillation, β_{ce}=0) still reaches 96.7% recovery but floors shallower (5.3%)—labels are not needed for validity, only for the last ∼2% of budget.

3.2 Function calling: a real-world capability, label-free

We extract single-turn function calling from Qwen3-4B [16] using BFCL prompts [13] (400 tasks; 391 yield clean tool calls from the base model; 320 train / 80 held out; 350,208 channels). No gold calls are used: the teacher is the model's own greedy tool call, and fidelity is *verbatim* self-match—stricter than BFCL's semantic scoring, since one diverging token fails the example.

Held-out fidelity reaches 77.5% at 50% of channels and 76.3% at 40% (Figure 3). Two observations matter. First, the *unmasked* adapted model self-matches only 86% on held-out prompts—masking is not the dominant loss; adapter drift under a 320-prompt training set is. Relative to that ceiling, the masked model

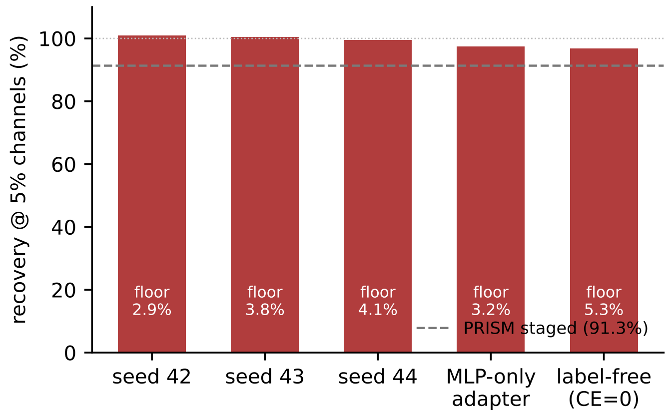

Figure 2: Recovery at 5% of channels across seeds and ablations; channel floors found by the controller annotated within bars.

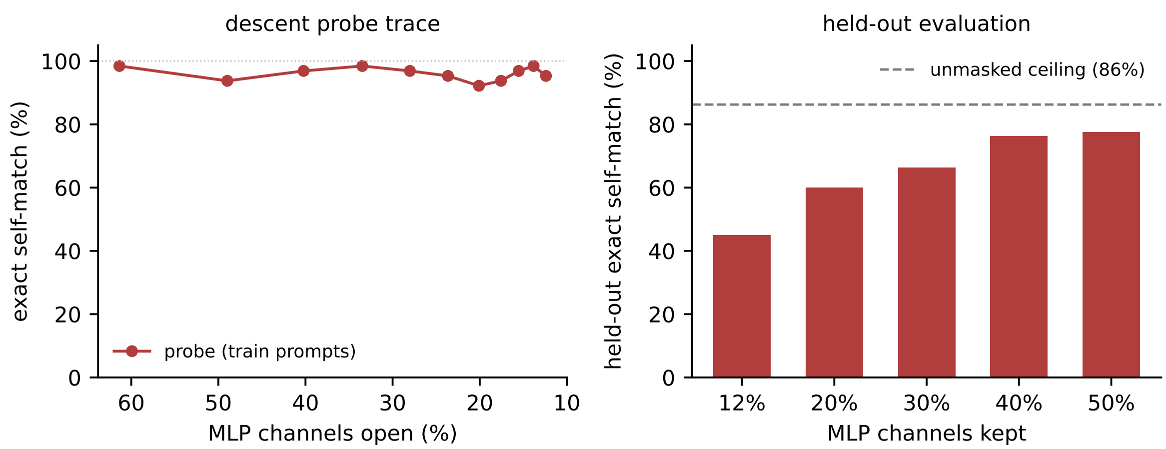

Figure 3: Function calling, Qwen3-4B. Left: probe trace during descent — exact self-match holds at 92–98% from 61% open down to 12% open on training prompts. Right: held-out self-match by budget; the dashed line is the unmasked adapted model's own self-match (86%), which upper-bounds every masked number.

retains ∼90% at 40–50% budgets. Second, the gap between probe fidelity on training prompts (92–98% down to 12% open) and held-out fidelity quantifies a generalization phenomenon that exhaustively-covered tasks like arithmetic cannot exhibit; prompt diversity, not extraction machinery, is the binding constraint. For calibration: PRISM's raw mask recovered 19.1% at 36.2% of channels on the 8B sibling model, and their best staged collimator (with a curated, schema-validated gold corpus and a multi-run objective search) reached 84.6%; metric and model-size differences make this directional context, not a head-to-head (§5).

3.3 Physical export: slicing is masking

For gated MLPs, a zero-masked channel contributes nothing to the down-projection; a deleted one contributes nothing by absence. The two are mathematically identical, so the masked evaluations above transfer to a physically smaller model. We verify: slicing Qwen2.5-Math-1.5B at the 2.9% floor yields 94.8% accuracy vs. 94.7% masked (one example in 810), and **1.54B → 0.42B parameters (3.66×)**. Generation speedup in our benchmark was only 1.11×: with 4-token outputs at batch 512, attention and the vocabulary projection dominate compute. Memory shrinks unconditionally; latency gains are workload-dependent—we report this plainly because slicing claims are routinely overstated.

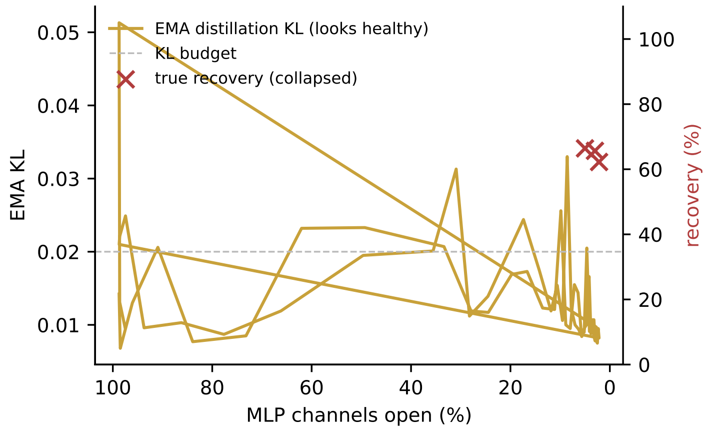

Figure 4: The failed KL-only controller (v2.0): EMA distillation KL stays under budget throughout the descent (left axis) while true held-out recovery collapses (crosses, right axis). Generation probes detect this; losses do not.

4 What fails, and what it implies

Distribution-level KL is miscalibrated at high sparsity. Our first controller used the KL budget alone to pace the descent. It descended to 2.2% open with EMA KL comfortably under budget—while true recovery collapsed to 62–66% (Figure 4). Teacher-forced divergence on answer tokens simply does not capture compounding autoregressive degradation. The fix—periodically *generating* and exact-matching—is cheap (seconds per probe) and is the load-bearing stability feature of the method. We believe this observation extends beyond extraction: any faithfulness or circuit-quality metric built on logit divergence under teacher forcing inherits the blind spot.

Layer-granular gates permit silent overfitting. Adding per-layer gates (hierarchical L_0, intended to enable wall-clock-friendly whole-layer drops) closed 16 of 28 layers; training KL stayed low because the adapter rerouted around missing layers *on the training distribution*, and held-out recovery collapsed to 66%. Channel-granular gates with all layers present do not exhibit this. Structured sparsity for speed should be grafted on after extraction, not learned jointly.

Engineering note. Stochastic gate sampling must occur outside gradient-checkpointed regions: in-region RNG produces a different recomputation graph and breaks backward. Pre-sampling per step costs nothing and composes with checkpointing, which 4B+ models require on consumer GPUs.

5 Honest comparison notes

Our arithmetic baseline reproduces PRISM's qualitative finding (attribution masks fail small: 0.3% recovery at 5%) but uses grad×act attribution and zero-isolation rather than their ReLP and counterfactual patching; PRISM's reported numbers come from their paper, not our re-runs. Zero-isolation is the harsher operator, so our baseline understates theirs. Function-calling comparisons differ in model size (4B vs. 8B) and metric (verbatim self-match vs. BFCL exact match); we present them as context. Arithmetic results are 3 seeds; function calling is a single seed. The function-calling floor was step-cap-limited, not probe-detected. PRISM's 10^3 H100-hour budget bought breadth (translation, an 8B model, extensive controls) that our \$5 did not; the claim here is that the *per-result* process cost collapses, not that our evidence is broader.

6 Related work

PRISM [12] defines the problem, contract, and metric we use; our method is their stated future direction, built. Learnable L_0 masks during adaptation descend from Louizos et al. [10] and movement pruning [14]; differentiable circuit discovery applies the same machinery to interpretability [2, 3]. Our distinction from both is the objective: a behavior-preservation contract with an unmasked-validity guardrail, rather than aggregate-loss compression or circuit faithfulness. Self-distillation into a substructure relates to on-policy and generalized knowledge distillation [8, 1], here with teacher and student sharing weights and the student differing only by a mask. Superposition [5] explains why attribution-only masks fail and adaptation is necessary; sparse autoencoders [4] and weight-sparse training [7] change the basis instead. The lottery-ticket line [6] seeks subnetworks that *retrain* to full performance; extraction requires the behavior to survive *without* retraining from initialization.

7 Limitations

Capabilities are bounded by prompt coverage: the function-calling generalization gap is a data problem the tool can flag (via held-out receipts) but not solve. Self-match undercounts semantic fidelity. The sliced artifact has heterogeneous layer widths and needs a re-slice step on load. We have not yet run the 8B headline, multi-seed function calling, or non-Qwen replications beyond CPU-level architecture tests; these are in progress and gate any stronger claims. Easier extraction also eases misuse (extracting narrow capabilities from safety-tuned models); receipts and intended-use documentation ship with the tool, mirroring PRISM's stance.

Reproducibility

All numbers in this paper were produced by the released library or the research scripts in the same repository, on one rented RTX 4090. The paper's figures regenerate from committed result JSONs via `paper/make_figures.py`.

References

[1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *ICLR*, 2024.

[2] Adithya Bhaskar, Alexander Wettig, Dan Friedman, and Danqi Chen. Finding transformer circuits with edge pruning. In *NeurIPS*, 2024.

[3] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *NeurIPS*, 2023.

[4] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv:2309.08600*, 2023.

[5] Nelson Elhage et al. Toy models of superposition. *Transformer Circuits Thread*, 2022.

[6] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *ICLR*, 2019.

[7] Leo Gao et al. Weight-sparse transformers have interpretable circuits. *arXiv:2511.13653*, 2025.

[8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.

[10] Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through l_0 regularization. In *ICLR*, 2018.

[11] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *ICLR*, 2017.

[12] Abhishek Mishra and Krishna Pagare. Prism: Unlocking language model capability extraction. In *1st Conference For AI Scientists (CAISc)*, 2026.

[13] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *ICML*, 2025.

[14] Victor Sanh, Thomas Wolf, and Alexander M. Rush. Movement pruning: Adaptive sparsity by fine-tuning. In *NeurIPS*, 2020.

[15] An Yang et al. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv:2409.12122*, 2024.

[16] An Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.